

Ejercicio

Lambda, decoradores,
generadores.

Python Intermedio





Lambda, decoradores, generadores

Este ejercicio práctico está diseñado para consolidar y aplicar los conceptos fundamentales de las **funciones para modularidad** que has estudiado en la Unidad 1. Al completarlo, habrás demostrado tu dominio en:

- Funciones lambda
- Funciones de orden superior
- Decoradores
- Generadores

Sistema de Monitoreo Ambiental Inteligente: Procesador de Alertas de Temperatura

Una empresa que desarrolla soluciones de monitoreo ambiental inteligente, necesita un pequeño módulo en Python para procesar datos de temperatura, recogidos por sensores ubicados en distintas ciudades. Este módulo debe ser capaz de filtrar, transformar, agrupar y analizar estos datos de forma eficiente y legible, usando funciones y estructuras modernas de Python.

El equipo de desarrollo quiere garantizar que el código sea modular, fácil de extender y eficiente en memoria, por lo cual, han solicitado que la solución sea implementada utilizando herramientas como lambda, funciones de orden superior, decoradores y generadores. Tu tarea será construir este módulo atendiendo a los **siguientes** requisitos:



- **Generador de datos:**

- Crea una función generadora *leer_temperaturas()* que devuelva, una a una, tuplas con el formato: ("Ciudad", temperatura)
 - ✓ Puedes simular los datos con una lista interna de ejemplo:
[("CDMX", 26), ("Monterrey", 34), ("Toluca", 19), ("Cancún", 38), ...]
 - ✓ El generador debe tener *yield* y no usar *return* directo.

- **Filtrado con *filter()* y *lambda*:**

- Filtra los registros que tengan temperaturas mayores o iguales a 30°C.

- **Transformación con *map()* y *lambda*:**

- Convierte cada tupla a un string con el formato:
"Alerta de calor en [Ciudad]: [Temperatura]°C"

- **Ordenamiento con *sorted()* y *key=lambda*:**

- Ordena los registros filtrados por temperatura en orden descendente.

- **Resumen con *reduce()*:**

- Calcula el promedio de temperaturas mayores a 30°C.
- Usa *reduce()* desde el módulo *functools*.



- **Decorador personalizado:**

- Crea un decorador llamado `@auditar_funcion` que:
 - ✓ Imprima el nombre de la función que se está ejecutando.
 - ✓ Cuente cuántas veces ha sido llamada.
 - ✓ Opcional: muestre la duración de ejecución (puede reutilizar el decorador `@medir_tiempo` visto en los recursos básicos)

- **Salida final esperada:**

- Imprimir la lista ordenada de alertas.
- Imprimir el promedio de temperaturas en formato:
"Temperatura promedio de alertas: 33.5°C"

¡Felicidades! Has concluido con éxito el ejercicio de Lambda, decoradores, generadores.